



Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

LAB 11 INSTRUCTIONS

For

Classify IPv4 scope and headers

Course Code: C1123

Conducted by

Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

Version 5.1

Lab 11 — Classify IPv4 scope and headers

Course: Wireshark Network Analysis Masterclass (C1123)

Topic: 11 — IPv4 Traffic Analysis

Learning outcome: LO3 — Interpret DNS, ARP, IPv4, ICMP and UDP evidence.

Time: about 50 minutes

Version: v5.1 · 5 October 2026

Scenario

The security team asks whether any unusual IPv4 traffic — multicast, broadcast or fragments — appears in the branch capture.

Goal

TTL limits forwarding hops; it is not a latency measurement.

What you will produce

A completed IPv4 header and scope sheet.

Before you start

- Wireshark 4.6 or later, with the TShark command-line tools on PATH.
- Python 3 for the verification and export scripts (Windows: use `py -3` in place of `python3`).
- Open this lab folder in a terminal; every file the lab needs is inside it.
- Use only the supplied synthetic captures — do not capture on a network you are not authorised to monitor.

Files for this lab

File	What it is for
data/branch-office.pcap	Synthetic branch-office capture used by the lab
assets/scenario.md	The help-desk ticket that sets the scenario
assets/checks.json	Filters and frame counts the fixture must satisfy
assets/expected-evidence.png	The packet list your filter should produce
scripts/verify.py	Checks the capture facts with TShark
scripts/export_evidence.py	Exports this lab's evidence rows to outputs/evidence.csv
outputs/findings.md	Findings sheet you complete
assets/ip-header.csv	IPv4 header sheet you complete

Lab at a glance

1. Record IPv4 header fields of an echo
2. Compare multicast MAC and IP destination
3. Check for fragmented packets
4. Classify scope in the IP header sheet

Step-by-step

1. Prepare your lab workspace. Open this lab folder in a terminal. Windows uses `py -3` in place of `python3`. Wireshark GUI alone is sufficient for the investigation; CLI scripts require TShark on PATH. Run the fixture verification first.

```
python3 scripts/verify.py
```

2. Open `data/branch-office.pcap`. Select an ICMP echo request. Expand IPv4 and record version, IHL, total length, TTL, protocol, flags and fragment offset.

3. Apply `ip.dst == 224.0.0.1` and compare Ethernet destination `01:00:5e:00:00:01` with IPv4 multicast destination. Record TTL 1.

4. Apply `ip.flags.mf == 1 || ip.frag_offset > 0`. Confirm no fragmented packets in this fixture; absence here does not demonstrate a universal MTU.

5. Complete `assets/ip-header.csv`. Explain why ARP broadcast is not an IPv4 broadcast packet, and why a capture at one point cannot estimate hop count from TTL without knowing initial TTL.

6. Export a reproducible packet table. Record the filter, frame numbers, measurement, explanation and limitation in `outputs/findings.md`.

```
python3 scripts/export_evidence.py
```

Expected evidence

With the lab filter applied, your packet list should match the frames below (produced by TShark from this lab's own capture).

ip.dst == 224.0.0.1					
No.	Time	Source	Destination	Protocol	Info
17	1.030000	192.0.2.10	224.0.0.1	UDP	55001 → 5000 Len=13
branch-office.pcap Displayed: 1					

Lab 11 expected evidence

Test it

The supplied `ip.dst == 224.0.0.1` expression matches 1 frame(s) in the specified capture. Run `scripts/verify.py` and compare the listed frame numbers. Keep your findings and exported table in `outputs/`. Explain the observed result rather than only copying a count.

Troubleshooting

- TShark not found: install Wireshark CLI tools and add the installation folder to PATH; on Windows use the Wireshark install directory.
- A filter returns zero: clear other filters, use the specified capture, and check the expression is in the display toolbar.
- TLS remains opaque: select the matching `lab-tls.keys` file by absolute path, reload, and remove a key-log preference from another lab.

Try it with TShark

The same evidence from the command line — run it from this lab folder:

```
tshark -n -r data/branch-office.pcap -Y "ip.dst == 224.0.0.1" -T fields  
-e eth.dst -e ip.dst -e ip.ttl
```

Challenge

Develop and explain an alternative filter.

Reflection

Which second observation point would strengthen your conclusion?

Extension (optional)

IP lab — study TTL and fragmentation in the traceroute trace from the Kurose & Ross labs.

Source: J.F. Kurose and K.W. Ross, Wireshark Labs

(gaia.cs.umass.edu/kurose_ross/wireshark.php).

Reset

Clear all display filters and return to the C1123-Analyst or Default profile. If you loaded a TLS key log, remove it from Preferences > Protocols > TLS. The supplied captures never need regenerating for the core lab.

Note: The same steps appear in the Learner Guide. The slides show only the scenario and a summary.